Task Scheduling for Autonomous Vehicles with Heterogeneous Processing via Integration of the CARLA Simulator and StarPU Runtime

Ericson Marquiere Reis Silva
Department of Computer Science
Pontificia Universidade Catolica de Minas Gerais
Belo Horizonte, Brazil
marquiere@unifei.edu.br

Henrique Cota de Freitas
Department of Computer Science
Pontificia Universidade Catolica de Minas Gerais
Belo Horizonte, Brazil
cota@pucminas.br

***Abstract*—Autonomous driving vehicles (ADV) rely on complex sensor systems and real-time decision making algorithms to function effectively. Efficient task scheduling is crucial for optimizing heterogeneous computing resources, which ADVs require to process data in real-time. This paper presents a novel approach that integrates the CARLA simulator, a widely used platform for testing and validating autonomous systems, with the StarPU runtime system that offers flexible task scheduling for heterogeneous hardware. By combining these two systems, the paper explores different scheduling strategies for ADV perception and localization tasks. The methodology combines CARLA's simulation capabilities and StarPU's dynamic resource allocation to evaluate task execution times. The results show that this integration provides a practical platform for testing and evaluating scheduling strategies in a realistic simulation environment, significantly enhancing ADV performance in sensor processing and task management.**

***Index Terms*—Autonomous Vehicles, Task Scheduling, CARLA Simulator, StarPU Runtime System, Heterogeneous Processing**

I. Introduction

Autonomous driving vehicles (ADV) are a reality that is being tested in Europe and countries such as China and the United States [1]. Although it is not possible to define a specific date for its full deployment with level-5 automation features and the substitution of non-autonomous vehicles, it is a consensus that it will be the next means of transportation. Maybe even owning a car would be outdated, as it would become a service, which has started to be called a robotaxi in some cities where it has been tested [2]. These vehicles are composed of many sensors and rely on machine-learning models to handle multiple tasks [3]. Efficiently managing the entire processing pipeline and machine resources is crucial in these driving systems.

Efficient task scheduling is essential for the ADV to achieve real-time system requirements with minimal use of available processing resources [4]. Without an optimized task scheduling system, ADVs could experience delays in processing data, making decisions, or controlling actuators, which could lead to unsafe situations [5]. Being a heterogeneous system, as most ADV prototypes use both CPU and GPU at least [1], complex scheduling techniques are required to manage the workload to be processed. Instead of manually drafting a heterogeneous task scheduling technique from scratch, a support library would come in handy to release the programmer from the burden of coding this part, allowing him to focus on the algorithmic part of the system in development. StarPu is a runtime system that works as a support library to schedule tasks in heterogeneous systems, working in conjunction with many different languages that offer optimized heterogeneous scheduling strategies [6]. It provides the necessary infrastructure to allocate tasks through multiple processing units, such as CPUs and GPUs, improving overall system performance.

Furthermore, an ADV system is necessary to apply the scheduling strategies, which could be a prototype or a simulated environment. As prototypes are complex and expensive to build, the use of simulators would be preferable for the fast testing, validation, and performance evaluation of the autonomous vehicle scheduling algorithms. A well-designed simulator is the CARLA [7], an open source platform for developing, testing, and validating autonomous driving systems. It offers researchers a realistic, highly detailed, safe, and controlled environment to validate their developments. However, CARLA lacks the tools to seamlessly manage task scheduling on heterogeneous computing resources.

Analyzing the ADV infrastructure, it can be seen that in the perception system, most of the tasks that occur simultaneously are concentrated there [1], [8]. Therefore, the ADV architecture would benefit the best from properly managing the computational resources. This paper aims to evaluate task scheduling strategies applied to task graphs generated from the organization of sensors and detection algorithms workload into joined tasks of complete perception micro modules (such as each camera and its related perception algorithm). Figure 1 shows the virtual setup based on the Kia Niro prototype [9].

This work was supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) under Grants PROAP 88887.842889/2023-00–PUC/MG, PDPG 88887.708960/2022-00–PUC/MG, and Finance Code 001; in part by the Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) under Grant 311697/2022-4; and in part by the Pontifícia Universidade Católica de Minas Gerais (PUC Minas) under Grant FIP 2024/30947.

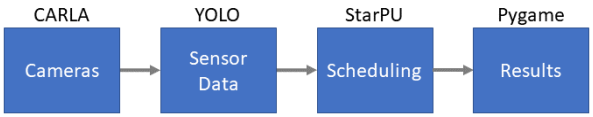

Fig. 1: System Overview

One of the main contributions of this paper is the integration of CARLA for realistic simulations and StarPU for efficient resource management to improve ADV task scheduling. To the best of our knowledge, this is the first study to explore that. The second contribution is the exploration of different strategies for task scheduling in a heterogeneous system comprising GPU/CPU processing units to improve task execution times and overall system performance. For this, we designed an ADV virtual model up to the sensor processing management by the CARLA simulator. The remainder of this paper is organized as follows: Section II presents Related Work, Section III is the Experimental Setup, Section IV shows the Results and Discussion, and finally, Section V describes our conclusions.

II. Related Work

A. Task Scheduling for Autonomous Vehicles

In the work by Balasekaran et al. [5], the authors propose an intelligent task scheduling mechanism for autonomous vehicles that uses deep learning algorithms. Their method focuses on reducing execution time and improving resource utilization using a single-layer feedforward neural network (SLFN) for task distribution. Tasks are scheduled using an Earliest Hyper Period First (EHF) scheduler, which helps minimize the task miss rate and energy consumption. This approach was validated on multicore System-on-Chip (SoC) platforms.

Dai et al. [10] present a scheduling algorithm for ADV tasks using mobile edge computing (MEC). The system offloads computationally intensive tasks to edge servers, considering that the ADV onboard computational systems are limited in battery life and computing capabilities. Their strategy uses an improved Earliest Deadline First (EDF) algorithm to manage task scheduling with task migration from servers. For this approach to work, the authors rely on the speed of the 5g wireless networks to improve data exchange between vehicles and servers.

Also dealing with edge servers and vehicular networks, Qi et al. [11] focus on parallel task scheduling, combining edge computing with deep reinforcement learning to improve task allocation. They developed a distributed multilevel task scheduling (MDTS) algorithm to assign tasks to edge nodes, prioritizing load balance and total execution cost. This method focuses on learning from the environment to dynamically allocate tasks across various processing nodes, including onboard processors and MEC servers.

Amarnath et al. [4] present a heterogeneity-aware task scheduling algorithm (HetSched) designed for SoCs in ADV. Dynamically prioritizes tasks based on the variation in execution time between different processing elements of the SoC. It prioritizes safe-critical tasks and trims the ones that are not essential and could not meet the requirements to improve the ADV mission time. It uses the STOMP scheduler platform [12] to evaluate its designed algorithm.

Furthermore, Xu et al. [13] propose the Age of Information (AoI)-centric task scheduling approach that aims to ensure that tasks use the most recent sensor data, which is critical for correct detection, planning, and action taking by ADV. The schedule uses a reinforcement learning solution by integrating its scheduler with a Double deep Q network (DQN) to solve hard combinatorial problems.

Existing approaches frequently face challenges such as rigid, platform-specific implementations that lack flexibility for heterogeneous systems. Additionally, theoretical scheduling models are often validated only in constrained environments, and high overhead from edge computing or complex reinforcement learning models makes them unsuitable for real-time applications.

In contrast, this work aims to build a scheme that could test any developed task scheduling plan by simply porting it to Python language in a common working environment. It will allow users to apply well-established or novel-designed scheduling strategies or develop their own, providing a practical, real-world platform to manage heterogeneous task scheduling. By exploring the possibilities of integrating simulation software with a runtime system for task scheduling, it aims to offer dynamic task management and resource allocation strategies in a realistic driving environment to achieve results as close as possible to those achieved with ADV prototypes.

B. Simulation/System Integration for Autonomous Vehicles

Rosique et al. [14] provide a comprehensive overview of the available simulators. They classified simulators into four main types: model-based development simulators, robotics simulators, game engine-based simulators, and AV-specific simulators. It was highlighted that perception subsystems, such as LIDAR, cameras, and RADAR, are essential to create high-fidelity simulation conditions. However, they identified shortcomings in computational efficiency and the processing of large-scale simulations.

Five years later, Li et al. [15] provided an updated and more detailed exploration of the evolution of simulators. It offers new classification categories for simulators: traffic flow, sensory data, driving policy, vehicle dynamics, and comprehensive simulators. This last category mainly includes all-in-one solutions that comprise all previous categories, yet it is mostly proprietary software. It is important to note the large number of new software deployed after the work of [14] in 2019. In [15], advances in simulation technology, particularly in high-fidelity environments and the inclusion of various driving conditions, were discussed to make simulators more relevant for testing complex driving policies and end-to-end systems. However, limitations in computational performance and experimental reliability were pointed out, particularly

when scaling simulations for real-world ADV applications. It also advocates that the key feature to get the simulation as close to the real world as possible is integrating real-time frameworks used in ADV prototypes such as ROS2 [16] and Autoware [17].

Connection to CARSIM [18] or Chrono [19] where vehicle controls and dynamics are handed to an external simulator and returned to CARLA only the state and location of the vehicle. Integration with Robot Operating System (ROS) [16] versions 1 and 2 takes advantage of CARLA high-definition maps to visualize the environment while keeping control of all other aspects such as spawning actors, sensors data processing, and vehicle control.

Furthermore, it does a co-simulation with Sumo [20] and PTV-Vissim [21], where the simulation is synchronized, and the user can explore and choose the capabilities of each simulator. However, there is no integration with any tools that would focus solely on improving execution performance or managing and distributing the tasks to be executed.

III. Experimental Setup

In this section, we describe the whole setup, starting from the vehicle deployment and the attachment of sensors using the CARLA simulator environment. The data of these sensors are processed to obtain meaningful information. This is done by organizing them into tasks composed of sensor and processing algorithms and sending them to the StarPU task management to apply scheduling techniques. The simulation runs on a machine with Ubuntu 20.04 as an operating system equipped with an i5-10400 processor with six cores, a Geforce RTX 3060 GPU, and 32GB of DDR4 RAM.

A. CARLA simulator

CARLA (Car Learning to Act) is an open-source simulation platform designed to facilitate the development, testing, and validation of autonomous driving systems [7]. It provides an urban environment where researchers can simulate complex driving conditions, including changes in weather, pedestrian movements, and vehicle behaviors. Built on Unreal Engine 4, CARLA offers high-fidelity 3D environments, making it suitable for academic research and industry applications.

CARLA's sensor suite, which includes cameras, LIDAR, radar, and GPS, permits developers to simulate a wide range of real-world conditions. This supports the testing of perception, localization, and control algorithms. Moreover, CARLA offers flexible API support, enabling integration with Python and C++ for logical interaction with autonomous driving stacks. The platform is also highly customizable, allowing users to modify the environment, sensor settings, and traffic scenarios, creating a rich testing ground for autonomous systems.

B. StarPU Runtime System

StarPU is a runtime system designed to schedule and manage tasks on heterogeneous computing platforms efficiently. Its main purpose is to enable the smooth execution of heterogeneous applications on hardware with varying architectures, such as multicore CPUs, GPUs, and other accelerators [6]. By abstracting the complexities of heterogeneous hardware, StarPU allows developers to focus on optimizing their algorithms without needing to manage the complexity of hardware-specific execution manually.

One of the key features of StarPU is its dynamic scheduling ability, where the system selects the most appropriate hardware resource, such as CPU, GPU, FPGA, and DSP, to execute each task based on the current system load and task requirements. This dynamic resource allocation helps improve overall system performance and task execution efficiency. StarPU achieves this through various scheduling policies, such as work-stealing, priority-based scheduling, and data locality optimization, which ensure that the workload is balanced across available resources while minimizing communication overhead [22].

C. Simulation Environment

In the CARLA simulator space, spawning is deploying something into the simulated world: sensors, infrastructure, pedestrians, or vehicles, which the API calls actors. Figure 2 shows a vehicle setup used in the simulations based on a Tesla Cybertruck composed of eight cameras that dynamically cover a field of view of 360 degrees.

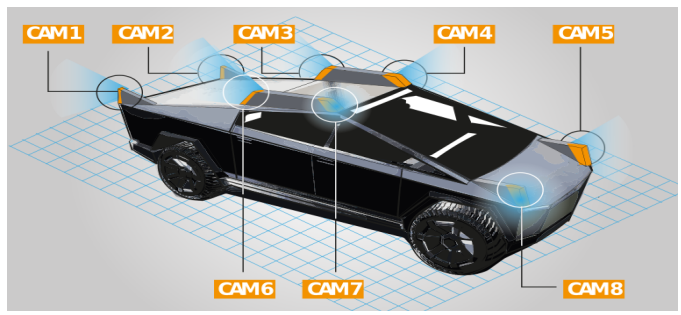

Fig. 2: Vehicle Setup

The Carla server was loaded with the Town10 map, a downtown urban suburb with residential buildings on its outskirts and skyscrapers in its central area. In addition, pedestrians and other vehicles were spawned on the map, and their movement was set to auto-mode to best simulate a city environment. In addition, the main vehicle was running in auto mode to continue moving around the city to record data using the sensors.

To visualize sensor information and see vehicle location, a PyGame [23] was deployed, generating a window subdivided into small sub-windows for each camera spawned in the vehicle to visualize processed data from all sensors. Figure 3 shows how the data were visualized.

D. Sensor Data Processing

In the RGB camera image, the Yolo algorithm (You Only Look Once) was utilized to detect objects in the provided camera feed. This algorithm was created in 2015 and quickly gained popularity due to its accuracy, high speed, and

Fig. 3: PyGame Visualization Window

lightweight. Using a convolutional neural network (CNN) to scan the entire picture simultaneously, it can process images at 45 frames per second and segment them into small boxes with meaningful identification. In this simulation, we used version 5 of the algorithm with a pre-trained database maintained by Ultralytics [24], which now hosts and maintains the algorithm. This version is simple and robust, compatible with Python 3.7 to 3.9, which is in the range of compatibility with the CARLA simulator. Yolo received 640x360 pixels images at 20 frames per second.

The sole objective of processing sensor data with complex algorithm is to generate enough workload to get meaningful results and investigate the advantages of applying the task scheduling techniques to an ADV processing pipeline in its heterogeneous system. Furthermore, the cameras were considered enough workload for the ADV's perception system. Hence, it did not include algorithms related to the planning and control parts of the ADV Architecture [1].

E. StarPU Integration

StarPU was integrated into the CARLA simulation using Python, a common language that allows communication between the two libraries through their APIs. This allowed us to manage and schedule the computational workload from the cameras and the Yolo algorithm.

The first step is to define the tasks that StarPU will manage. Each task was defined as composed of raw data and an algorithm to process these data. For that to happen, the sensor data and its processing algorithm must be defined within a Python function. This function becomes a task. This task can be submitted using a decorator for the function or explicitly submitting it with a StarPU submit task command. Despite that, other workloads from the simulation, such as vehicle control and Pygame visualization windows, were not sent to StarPU management.

StarPU can dynamically schedule tasks and manage their distribution through the GPU and CPU processing nodes, following a scheduling strategy defined inside the code or as an option at the time of executing the script. It offers a range of strategies that can be quickly changed every time the script is executed using the option STARPU_SCHED. The Python time library was used to measure the execution time for each task and the iteration time of the full simulation.

IV. Results and Discussion

A. StarPU integration into CARLA simulation

The integration of StarPU with the CARLA simulator demonstrated an interesting way to manage the computational workload in ADV. This paper focused on managing sensor processing for object detection tasks, but this approach could be extended to the entire ADV processing pipeline, including navigation and vehicle control.

Using StarPU, it was possible to submit vehicle processing tasks as Python functions simply by using the function *starpu.task.submit*. It is important to note the relevance of the library *asyncio* for the proper execution of StarPU as it gives the necessary flexibility of asynchronous execution, which is required when programming in a task-oriented manner. In addition, it allowed task submission parallelism without relying on multiple threads since, in Python, they are limited by the Global Interpreter Lock (GIL), preventing the language from having real parallelism.

As seen in Figure 3, during the simulation, a real-time display of the task processing times individually and the total iteration time was created using PyGame. This feedback allowed for the verification of task scheduling performances during vehicle navigation on the map in autopilot. Hence, it was possible to identify performance bottlenecks and the impact of different scheduling strategies.

B. Application of Scheduling Strategies

We evaluated the impact of different scheduling strategies on the performance of the ADV sensor processing pipeline using several scheduling algorithms provided by StarPU. The recorded times are presented in this section. The scheduling strategies used in this paper are as follows:

- **WS**: Work-Stealing
- **LWS**: Local Work-Stealing
- **PRIO**: Priority-based
- **RANDOM**: Random task allocation
- **HETEROPRIO**: Heterogeneous Priority
- **EAGER**: Eager scheduling
- **DMDA**: Deque Model Data Aware
- **PHEFT**: Parallel Heterogeneous Earliest Finish Time
- **PEager**: Parallel Eager scheduling
- **OSOnly**: The operating system handles tasks

The scheduling strategies helped us identify performance differences in the ADV sensor processing pipeline. Every scheduling strategy was running for 2000 full code iterations. The following results summarize the performance of various scheduling strategies in terms of Average Iteration Time, Average Task Time, CPU Usage, and GPU Usage.

It is important to note that, although StarPU was responsible for managing the workload of the cameras and their processing algorithms, the performance metrics were measured across the entire simulation loop.

1) Iteration Time: The average iteration time is a critical metric of the ADV that represents the overall speed of the simulation loop. A lower iteration time indicates a more efficient

TABLE I: Performance for Different Scheduling Strategies

Scheduling Strategy	Avg Iteration Time (sec)	Avg Task Time (sec)	Avg CPU Usage (%)	Avg GPU Usage (%)
WS	0.0777	0.0266	29.8274	80.0350
LWS	0.0697	0.0478	35.7746	84.0919
PRIO	0.0795	0.0195	27.3814	78.1834
Random	0.0716	0.0374	33.4894	84.3138
HeteroPrio	0.0709	0.0466	35.1347	85.2222
Eager	0.0774	0.0196	28.1366	79.0994
DMDA	0.0690	0.0475	35.5200	85.2729
PHEFT	0.0785	0.0227	29.2147	79.3250
PEager	0.0781	0.0188	27.8244	78.5460
OSOnly	0.0804	0.0154	26.6940	77.5979

scheduling strategy to process simulation steps faster, which is essential for real-time autonomous vehicle applications. The items created in CARLA, such as vehicles, pedestrians, and cameras, are not included in the simulation loop, nor is the visualization in the Pygame window, which is also outside of the StarPU processing tasks.

Table I shows that the DMDA scheduling strategy achieved the lowest average iteration time, 0.06899 seconds, outperforming all other strategies. This suggests that DMDA effectively balances the workload between CPU and GPU resources, minimizing idle times and overhead as a data-driven scheduling strategy. It also requests that the task performance model be configured as an option in the StarPU task submit function to work correctly in the StarPU environment. Following DMDA, the LWS and HeteroPrio strategies also shows competitive average iteration times of 0.06966 seconds and 0.07092 seconds. In contrast, the OSOnly script exhibits the highest average iteration time of 0.08038 seconds.

Further analyses can be performed by verifying the evolution of iteration time over iterations, as seen in Figure 4. It confirms the good results of DMDA, HeteroPrio, and LWS while maintaining a low iteration with feel spikes indicating better balance and resource utilization. These strategies, in many points, can reduce the iteration time up to 40% of the average. PRIO and PEager also show promising results but with more oscillations, which affects its overall performance. OSOnly tends to have the most stable but higher iteration time, as seen in Table I, which is expected due to the nonexistence of StarPU scheduling strategy since the operating system handles tasks. Random shows good results but more frequent and unpredictable spikes, which indicates it's difficult to consistently allocate tasks over time.

2) Task Time: The average task time measures the time to execute individual tasks within the simulation. Again, the only tasks submitted are related to the cameras' image processing. The average recordings could be verified in Table I and also in Figure 5. It could be seen that the scheduler needed some time to settle up and stabilize its execution of above 100 iterations. In addition, the constant oscillation in the average task time could be due to the constant variability of the cameras' input data, as the collected data varies significantly depending on

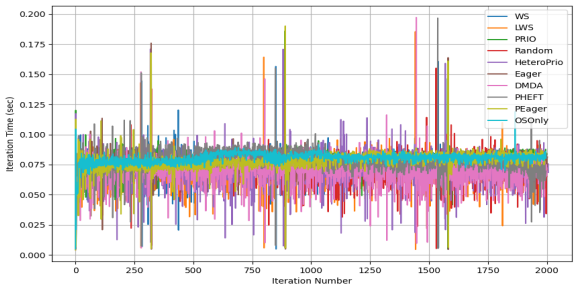

Fig. 4: Iteration Times over Iterations

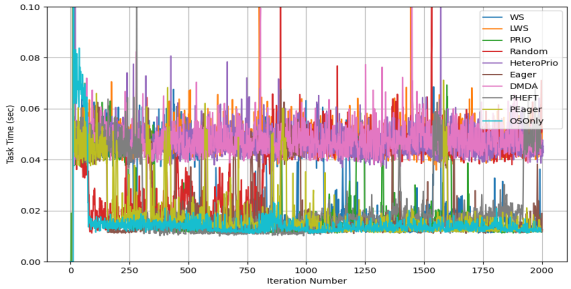

Fig. 5: Task Times over Iterations

the number of objects detected.

It could also be verified that the scheduling policies are separated into two groups. WS, PRIO, Eager PHEFT, and OSOnly have small task times compared to LWS, Heteroprio, and DMDA. However, this did not necessarily translate into overall performance. For example, the OSOnly strategy recorded the lowest average task time of 0.01543 seconds. However, its longer iteration time proved inefficiencies at the simulation loop level, possibly because of the lack of optimization of the utilization of computing resources. Similarly, the PEager and PRIO strategies also showed low average task times of 0.01882 seconds and 0.01948 seconds but did not achieve the highest performance.

StarPU is a runtime system that introduces overhead and its scheduling performance improves with increasing workloads. The better task time performance of OSOnly can be explained by the absence of this overhead. In this paper, we conducted a controlled experiment to validate the CARLA/StarPU integration under scheduling evaluation. As discussed in future work, larger workloads will be evaluated to propose a new scheduling strategy.

3) Resource Utilization: Efficient utilization of CPU and GPU resources is crucial for optimizing performance in a heterogeneous computing system. Higher resource usage typically leads to better performance, provided the resources are appropriately applied. As can be seen in Figure 6, the DMDA strategy not only achieved the lowest iteration time but also had the highest GPU usage at 85.25 % and the second highest CPU usage at 35.52 %. This utilization indicates that

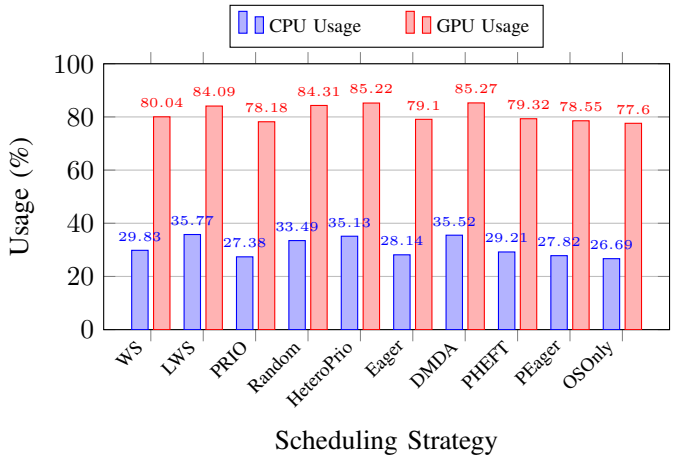

Fig. 6: CPU and GPU Usage for Each Scheduling Strategy

DMDA effectively explores CPU and GPU resources, leading to improved performance. The HeteroPrio and LWS strategies perform close to DMDA performance, showing that these are the options for heavy GPU workloads.

On the other hand, the OSOnly strategy had the lowest CPU (26.69 %) and GPU (77.60 %) usages, aligning with its higher iteration time. This suggests that OSOnly does not fully explore the available hardware resources, leading to less efficient simulation performance.

V. Conclusions

This paper successfully integrated the CARLA simulator and the StarPU runtime system, exploring various scheduling strategies for autonomous vehicles. This integration demonstrated the potential for optimizing perception tasks in a heterogeneous computing environment applied to ADV. However, the study is limited to simulated sensor data that could have discrepancies compared to real world results. Despite the limitations, results showed that the dynamic allocation of tasks across CPUs/GPUs significantly improved task execution efficiency. The DMDA scheduling strategy achieves the best overall performance despite the higher processing time per task.

Future work focuses on increasing the number of sensors and data processing algorithms, covering the other parts of the ADV pipeline, e.g., localization, decision-making, and control. Additionally, advances in high-performance computing hardware, such as specialized AI accelerators, are expected to open new possibilities for improving task execution efficiency including the integration of machine learning-based adaptive scheduling algorithms.

References

[1] E. Silva, F. Soares, W. Souza, and H. Freitas, "A systematic mapping of autonomous vehicle prototypes: Trends and opportunities," *IEEE Transactions on Intelligent Vehicles*, pp. 1–27, 2024.

[2] M. Kanbur, O. P. C, and P. Kulkarni, "Project risk mitigation in autonomous vehicles systems: The tesla way," in *2023 IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE)*, 2023, pp. 1–9.

[3] W. Jun, M. Son, and S. Lee, "Performance analysis and enhancement of multi-task learning for autonomous driving," in *2024 IEEE International Conference on Consumer Electronics (ICCE)*, 2024, pp. 1–4.

[4] A. Amarnath, S. Pal, H. T. Kassa, A. Vega, A. Buyuktosunoglu, H. Franke, J.-D. Wellman, R. Dreslinski, and P. Bose, "Heterogeneity-aware scheduling on socs for autonomous vehicles," *IEEE Computer Architecture Letters*, vol. 20, no. 2, pp. 82–85, 2021.

[5] G. Balasekaran, S. Jayakumar, and R. Pérez de Prado, "An intelligent task scheduling mechanism for autonomous vehicles via deep learning," *Energies*, vol. 14, no. 6, 2021. [Online]. Available: https://www.mdpi.com/1996-1073/14/6/1788

[6] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, "StarPU: a unified platform for task scheduling on heterogeneous multicore architectures," *Concurrency and Computation: Practice and Experience*, vol. 23, no. 2, pp. 187–198, 2011. [Online]. Available: https://inria.hal.science/inria-00550877

[7] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, S. Levine, V. Vanhoucke, and K. Goldberg, Eds., vol. 78. PMLR, 13–15 Nov 2017, pp. 1–16. [Online]. Available: https://proceedings.mlr.press/v78/dosovitskiy17a.html

[8] W. Zong, C. Zhang, Z. Wang, J. Zhu, and Q. Chen, "Architecture design and implementation of an autonomous vehicle," *IEEE Access*, vol. 6, pp. 21 956–21 970, 2018.

[9] M. Reke, D. Peter, J. Schulte-Tigges, S. Schiffer, A. Ferrein, T. Walter, and D. Matheis, "A self-driving car architecture in ros2," in *2020 International SAUPEC/RobMech/PRASA Conference*, 2020, pp. 1–6.

[10] H. Dai, X. Zeng, Z. Yu, and T. Wang, "A scheduling algorithm for autonomous driving tasks on mobile edge computing servers," *Journal of Systems Architecture*, vol. 94, pp. 14–23, 2019. [Online]. Available: https://doi.org/10.1016/j.sysarc.2019.02.004

[11] Q. Qi, L. Zhang, J. Wang, H. Sun, Z. Zhuang, J. Liao, and F. R. Yu, "Scalable parallel task scheduling for autonomous driving using multi-task deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 11, pp. 13 861–13 874, 2020.

[12] A. Vega, A. Amarnath, J.-D. Wellman, H. Kassa, S. Pal, H. Franke, A. Buyuktosunoglu, R. Dreslinski, and P. Bose, "Stomp: A tool for evaluation of scheduling policies in heterogeneous multi-processors," *arXiv preprint arXiv:2007.14371*, 2020.

[13] C. Xu, Q. Xu, J. Wang, K. Wu, K. Lu, and C. Qiao, "Aoi-centric task scheduling for autonomous driving systems," in *Proceedings - IEEE INFOCOM*, vol. 2022-May. Institute of Electrical and Electronics Engineers Inc., 2022, pp. 1019–1028.

[14] F. Rosique, P. J. Navarro, C. Fernández, and A. Padilla, "A systematic review of perception system and simulators for autonomous vehicles research," *Sensors (Switzerland)*, vol. 19, 2019.

[15] Y. Li, W. Yuan, S. Zhang, W. Yan, Q. Shen, C. Wang, and M. Yang, "Choose your simulator wisely: A review on open-source simulators for autonomous driving," *IEEE Transactions on Intelligent Vehicles*, 2024.

[16] Open Source Robotics Foundation, "Robot Operating System (ROS)," https://www.ros.org/, 2023, accessed: 2023-12-09.

[17] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. IEEE, 2018, pp. 287–296.

[18] Mechanical Simulation Corporation, "CarSim: Vehicle Dynamics Simulation Software," https://www.carsim.com/, 2023, accessed: 2023-12-09.

[19] University of Wisconsin-Madison, "Project Chrono: Physics-based Simulation Engine," https://projectchrono.org/, 2023, accessed: 2023-12-09.

[20] German Aerospace Center (DLR), "SUMO: Simulation of Urban Mobility," https://sumo.dlr.de/docs/index.html, 2023, accessed: 2023-12-09.

[21] PTV Group, "PTV Vissim: Traffic Simulation Software," https://www.ptvgroup.com/en/products/ptv-vissim, 2023, accessed: 2023-12-09.

[22] StarPU Team, *StarPU Handbook: A Runtime System for Heterogeneous Multicore Architectures*, 2023, accessed: 2023-12-09. [Online]. Available: https://files.inria.fr/starpu/doc/starpu.pdf

[23] S. Kelly, *Basic Introduction to PyGame*. Berkeley, CA: Apress, 2016, pp. 59–65. [Online]. Available: https://doi.org/10.1007/978-1-4842-2517-2_9

[24] Ultralytics, *Ultralytics Documentation*, 2024, accessed: 2024-09-12. [Online]. Available: https://docs.ultralytics.com/